小车组编队技术总结
背景介绍
小车
UWB
进入正题
编程语言
模式选择
姿态模型建立
数据读取
数据解码和提取
模型的代码实现
控制算法--加入pid

小车组编队技术总结

背景介绍

小车

- 我们有从rc遥控车改装而来的小车，并能利用树莓派和网络技术实现低延时的LAN和WAN远程控制由于本文聚焦于编队，所以这一块不在此赘述

UWB

- 硬件组成和操作
  - 具体请查看用户手册，但在这里还是进行简要的说明，我们先使用客户端对Linktrack系统根据需要进行配置，然后利用不同语言的串口读取模块进行定制化读取，比如我们在python中使用的是它的pyserial模块
- 技术特点
  - UWB(ultra wideband)是一种无线载波通讯技术，它不采用正弦载波，而是利用纳秒级的非正弦波窄脉冲传输数据，因此其所占的频谱范围很宽，那么随之而来的便是系统复杂度低，发射信号功率谱密度低，对信道衰落不敏感，截获能力低，定位精度高等优点，尤其适用于室内等密集多径场所的高速无线接入等优点
- 厂商
  - 我们并不是完全从底层利用该技术，而是利用其它厂商的UWB产品，我们可以利用它来获得我们需要的数据，如距离，角度，速度，加速度等各种物理参数，从而来实现我们的编队应用，我们使用的是深圳空循环科技有限公司的LinkTrack系统，关于该产品的特性，具体可查看官网https://www.nooploop.com/，其中有详细的使用说明和技术手册，甚至还有根据他的“NLink”协议构建的开源的C语言源代码https://github.com/nooploop-dev/nlink_unpack

进入正题

编程语言

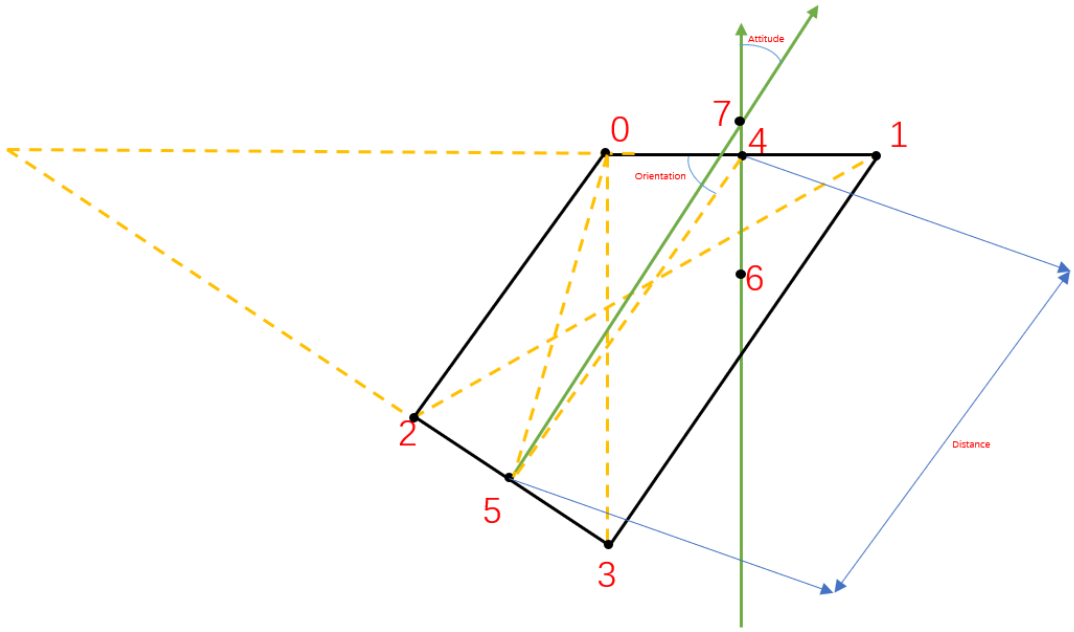
- 此项目使用python编程，将不同的模块分装进入类，最终在main.py中调用即可

模式选择

- 这里我们选择的是DR(distributed ranging)，关于不同的模式，请参见用户手册

- 选择这个模式是因为我的师兄骆洋当时提供了三个方向（详情参见骆洋编队算法解决方案），而我们所使用的前二后二模型（下面会讲到）是其中最简洁的一种，并且只需要UWB提供距离即可

姿态模型建立

- 

如图所示是我们所采用的前二后二编队模型，
其中，节点0，1，2，3分别代表四个UWB数据采集模块，
通过$DRmode$我们可以获得L_{01}，L_{23}，L_{02}，L_{03}，L_{12}，L_{13}之间的距离
而L_{01}，L_{23}分别表示前车和后车，是定长，在读取UWB数据时跳过
下面将用”$L+num$”的方式来表示一条特定边的长度接下来将给出姿态偏移（$Attitude$），
偏移距离（$Distance$），引导方向（$Orientation$）三个量的推导过程

  - Attitude

    延长L_{10}和L_{32}，则此时$Attitude$利用简单的余弦定理便能求得

    $$Attitude \in [-\pi, \pi]$$

    我们规定，当角度在竖轴右侧时为正值，左侧时为负值，下同
    当$Attitude \in [0, \pi]$，上述延长线交点位于左，即前车位于后车的左前方，应左转，反之相反

    - $$\angle 201 = \arccos(\frac{L_{02}^2 + L_{01}^2 - L_{12}^2}{2 * L_{02} L_{01}})$$
      $$\angle 023 = \arccos(\frac{L_{02}^2 + L_{23}^2 - L_{03}^2}{2 * L_{02} L_{23}})$$
      $$Attitude = \pi - (\angle 201 - \angle 023)$$

  - Distance

    接下来计算偏移距离，也就是L_{45}

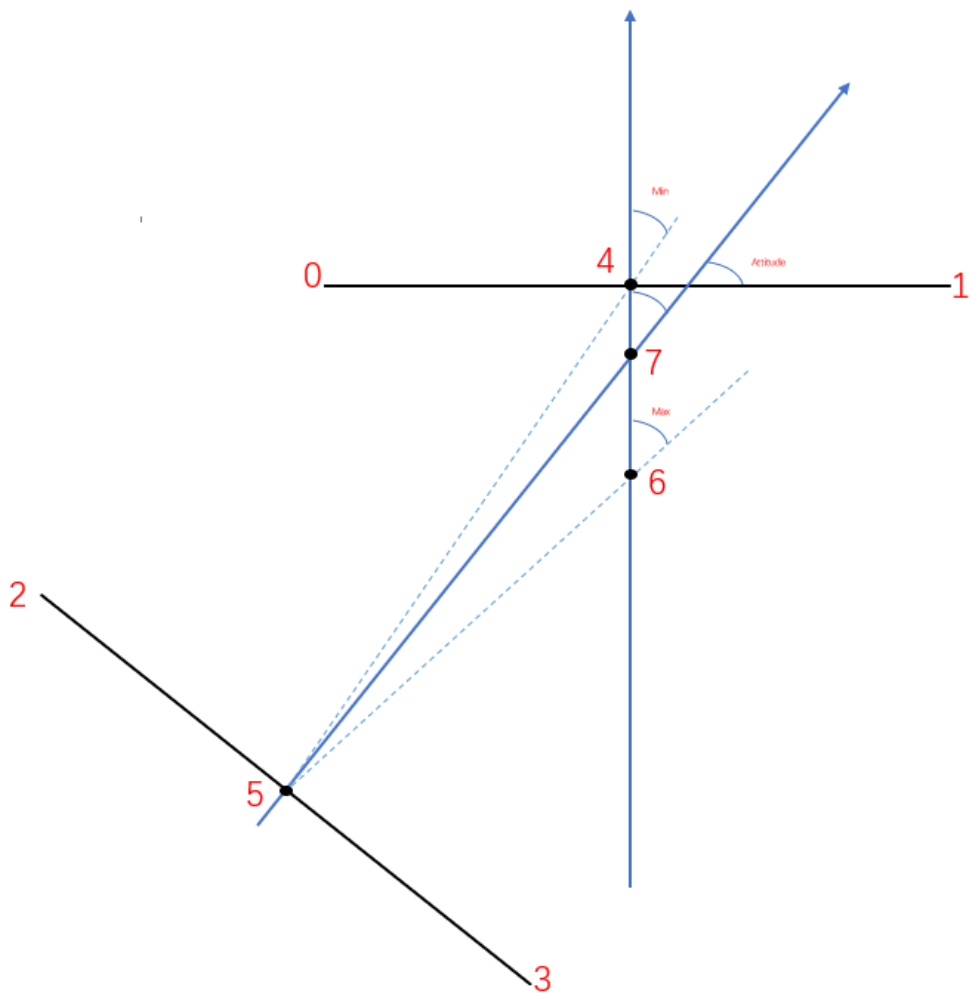
    - $$\cos(\angle 023) = \frac{L_{02}^2 + L_{25}^2 + L_{05}^2}{2 * L_{02} * L_{25}}$$
      $$L_{05} = (L_{02}^2 + L_{25}^2 - 2 * L_{02} * L_{25} * \cos(\angle 023))^{0.5}$$
      $$\angle 205 = \arccos(\frac{L_{02}^2 + L_{05}^2 + L_{25}^2}{2 * L_{02} * L_{05}})$$
      $$\angle 504 = \angle 201 - \angle 205$$
      $$Distance = L_{45} = (L_{05} + L_{04} - 2 * L_{05} * L_{04} \cos(\angle 504))^{0.5}$$

  - Orientation

- $$Orientation = \angle 045 = \arccos(\frac{L_{04}^2 + L_{45}^2 + L_{05}^2}{2 * L_{04} * L_{45}})$$

-

以上的姿态模型是最初的工作成果，但是在实际测试后，
我们发现为了实现编队跟随不应该把跟随点设置在前车上，
而应该是前车正后方，
于是在上述模型的基础上有了下列改进，
并增加跟随距离（$ChasingDis$）和跟随角$(ChasingAngle)$两个参数
下图中节点6即为追随点



  - ChasingDis和ChasingAngle
    -

$$\text{先说一下特殊情况，当}\angle 045=\frac{\pi}{2}\text{时，}\triangle 456\text{不存在，}$$
$$\text{此时令}ChasingDis=Distance-L_{46},$$
$$ChasingAngle=Attitude$$
$$\text{当}\triangle\text{存在时}$$
$$ChasingDis\text{始终是}L_{56}$$
$$\text{而对于}ChasingAngle,\text{我们需要进行分类讨论，首先看}\angle 045\in[0,\frac{\pi}{2}]\text{的情况}$$
$$\begin{cases} Attitude\in[0,max],|ChasingAngle|=|Attitude-max|=max-Attitude \\ Attitude\in[max,\pi],|ChasingAngle|=|Attitude-max|=Attitude-max \\ Attitude\in[-\pi,0],|ChasingAngle|=|Attitude-max|=|Attitude|+|max| \end{cases}$$
$$\text{则}ChasingAngle=Attitude-max$$
$$\text{符号原则见上述}Attitude$$
$$\text{对}Orienation\in[\pi/2,\pi]\text{使用同样的分析方法，不难发现此时}$$
$$ChasingAngle=Attitude+max$$
$$\therefore ChasingAngle=Attitude-\frac{|Orientation-\frac{\pi}{2}|}{\frac{\pi}{2}-Orientation}*max$$

数据读取

- 读取原理
- UWB采用串口通信，空循环的这套LinkTrack系统支持USB—C口协议,我们通过数据线将测量模块和树莓派相连，并使用python自带库pyerial(下载时使用pip install pyserial但导入时使用import serial)进行读取，UWB的刷新率为0-200Hz,因为编队对实时要求较高，所以我们采用最高的刷新率200Hz.
- 并行读取
  - 树莓派需要同时读取两个串口的数据，所以在此处需使用多线程来实现同步，但有一点需要注意，我们是一边读取一边使用，如果使用同一块内存，会有线程安全的问题，所以查资料决定此处使用列表存储读取数据，在这种情况下可以保证线程安全
- 帧丢失和短缺
  - 实际实验时发现存在帧丢失的情况，如一个帧应为159Bytes,但存在只有159-13*nBytes的情况，而13恰好是一个节点的数据长度，此时帧头出现在了数据中间的某个位置，需要说明的是，实验发现，这种情况占少数，但如果没有被发现，那么这种误差是具有传递性的，将导致读取到的数据一直是错误的，于是在这里需要有相应的补救机制，**即假设当前错误帧中帧头所代表的帧是完整的，那么在下一次以帧长度读取之前，先读取残余帧**
- 防止内存爆炸
  - 如果一直读取，会导致列表无限变长，最终可能会爆栈，所以要**时用时删**

此模块代码如下

- ```
  # writer: lihaonan_Bill
  # date:  2022/3/10 20:14

  """
  同时从两个UWB读取数据
  包含函数
  setup():
  read():在内部被装入线程池，不作为外部调用
  start():
  close():
  getData():
  ```
  ```

```
缺点：没有进程结束机制，只能在命令行中敲入 ctrl C强行退出
"""

import threading

# import time
import serial
import PortSearch


class ReadUWB:
    """
    serial初始化设置
    """
    threads = []  # 线程池，用来同时从两个串口读取数据
    """此处需要加线程锁"""
    effectiveData0 = []  # 存储有效数据(str形式)
    effectiveData1 = []
    getDataNum = 0
    # lock = threading.Lock()
    """
    在Python基本数据类型中list、tuple、dict本身就是属于线程安全的，
    所以如果有多个线程对这3种容器做操作时，我们不必考虑线程安全问题。
    """
    bps = 921600
    port0, port1 = PortSearch.getPort()
    ser0 = serial.Serial(port0, bps, timeout=1, bytesize=8)  # 串口应该根据实
际情况修改
    ser1 = serial.Serial(port1, bps, timeout=1, bytesize=8)
    flag0 = ser0.is_open
    flag1 = ser1.is_open

    @classmethod
    def setup(cls):
        if cls.flag0 and cls.flag1:
            print("port0 already open!")
            print("port1 already open!")
        else:
            cls.ser0.open()
            cls.ser1.open()

        """清除之前的数据，我们只需要当前的数据"""
        cls.ser0.flushInput()
        cls.ser1.flushInput()

    """
    UWB数据读取函数
    exception not written yet
    """

    @classmethod
    def read(cls, ser, effectiveData):
        # start = time.time()
        # count = 10000
        num = 0
        WrongNumber = 0
        print(ser.port, "starting...")
```

```
        while True:

            data_bytes = ser.read(159)
            data_str = data_bytes.hex()
            # print("get 159 bytes", count)
            # print(data_str, "\n")
            """
            采取偏移补救措施，最大化地利用数据
            """
            if data_str.startswith("5504"):

                effectiveData.append(data_str)
                num += 1
                # print(ser.port, "startswith\"5504\"", num)
            else:
                if data_str.count("5504") >= 1:
                    location = data_str.find("5504")
                    ser.read(location // 2)
                    WrongNumber += 1

    """
    进行线程操作
    """

    @classmethod
    def start(cls):
        th0 = threading.Thread(target=cls.read, args=(cls.ser0,
cls.effectiveData0,))
        th1 = threading.Thread(target=cls.read, args=(cls.ser1,
cls.effectiveData1,))
        cls.threads.append(th0)
        cls.threads.append(th1)
        for tt in cls.threads:
            tt.start()

    """
    关闭串口
    """

    @classmethod
    def close(cls):
        for tt in cls.threads:
            tt.join()
        cls.ser0.close()
        cls.ser1.close()

    @classmethod
    def getData(cls):
        """
        先判断数据是否存在，因为UWB的数据读取不是可控的
        若存在：获取字符串并清除
        :return: 一个含有两个字符串的列表
        """
        if len(cls.effectiveData0) > 1 and len(cls.effectiveData1) > 1:
            temp = [cls.effectiveData0[cls.getDataNum],
cls.effectiveData1[cls.getDataNum]]
            cls.effectiveData0.remove(cls.effectiveData0[0])
            cls.effectiveData1.remove(cls.effectiveData1[0])
```

```
            return temp
        else:
            return []
```

数据解码和提取

- 这一部分比较简单，查看Nlink协议，根据每一帧的组成按需提取需要的数据，当然，在那之前需要用客户端nassistant查看帧，其中，关于节点部分是可变的，意味着如果某个节点的数据没有被收到或增加了某个节点的数据，那么整个帧的长度都将改变，初步推断有时候帧长会改变就是因为这个原因

```
# writer: lihaonan_Bill
# date:  2022/3/11 20:42
"""
               front car
node0  -------------------------node1
-                                 -
-                                 -
-                                 -
-                                 -
-                                 -
-                                 -
-                                 -
node2  --------------------------node3
              rear car
e.g. L01,L13分别表示node0到node1的距离，node1到node3的距离，其他的距离表示类似
----------------------*
函数功能：在318长度的字符串中解析出我们需要的距离数据
数据单位：cm
"""

import threading


class Decode:
    lock = threading.Lock()
    """
    原始数据区：
    每个列表存放两个数据，一个是长度的累加和sum(cm)，另一个是累加次数，
    当累加次数到达5-10时，取平均值，数据取出并清零
    """
    L02_Origin = [0, 0]
    L12_Origin = [0, 0]
    L03_Origin = [0, 0]
    L13_Origin = [0, 0]

    """
    有效数据区：
    存放取均值过滤后的数据
    注意：该数据需要加线程所，因为除了本类相关线程在存放数据，还有Analyse类来访问数据
    """
    L02 = []
    L12 = []
    L03 = []
```

```
    L13 = []

    """
    函数功能：在318长度的字符串中解析出我们需要的距离数据
    """

    @classmethod
    def getData(cls):
        """
        若存在数据，则返回数据，然后清除
        :return: 返回由三个长度组成的列表
        """
        if len(cls.L02) > 1 and len(cls.L12) > 1 and len(cls.L03) > 1 and
len(cls.L13) > 1:
            temp = [cls.L02[0], cls.L12[0], cls.L03[0], cls.L13[0]]
            cls.L02.remove(cls.L02[0])
            cls.L12.remove(cls.L12[0])
            cls.L03.remove(cls.L03[0])
            cls.L13.remove(cls.L13[0])
            return temp
        else:
            return []

    @classmethod
    def decode(cls, data_str: str):
        frequency = 1  # 设置均值过滤频率
        if data_str[11] == "3":
            """这里因为是同步的，所以判断其中一条边长的累加值即可"""
            if cls.L03_Origin[1] == frequency:
                """
                算出平均值后L03和L13的累加值的平均值
                移交给有效数据区
                然后清零
                """

                cls.L03.append(cls.L03_Origin[0] / frequency)
                cls.L13.append(cls.L13_Origin[0] / frequency)
                cls.L03_Origin[0] = 0
                cls.L03_Origin[1] = 0
                cls.L13_Origin[0] = 0
                cls.L13_Origin[1] = 0

            else:
                """
                将16进制数转为十进制后存入相原始区
                """
                temp03: str = data_str[246:248] + data_str[244:246] +
data_str[242:244]
                cls.L03_Origin[0] += int(temp03, 16) / 10
                cls.L03_Origin[1] += 1
                temp13: str = data_str[272:274] + data_str[270:272] +
data_str[268:270]
                cls.L13_Origin[0] += int(temp13, 16) / 10
                cls.L13_Origin[1] += 1
        elif data_str[11] == "2":
            """这里因为是同步的，所以判断其中一条边长的累加值即可"""
            if cls.L02_Origin[1] == frequency:
                """
```

```
                算出平均值后L02和L12的累加值的平均值
                移交给有效数据区
                然后清零
                """

                cls.L02.append(cls.L02_Origin[0] / frequency)
                cls.L12.append(cls.L12_Origin[0] / frequency)
                cls.L02_Origin[0] = 0
                cls.L02_Origin[1] = 0
                cls.L12_Origin[0] = 0
                cls.L12_Origin[1] = 0

            else:
                """
                将16进制数转为十进制后存入相原始区
                """
                temp02: str = data_str[272:274] + data_str[270:272] +
data_str[268:270]
                cls.L02_Origin[0] += int(temp02, 16) / 10
                cls.L02_Origin[1] += 1
                temp12: str = data_str[298:300] + data_str[296:298] +
data_str[294:296]
                cls.L12_Origin[0] += int(temp12, 16) / 10
                cls.L12_Origin[1] += 1
```

模型的代码实现

- 这一块就只是将上面的“姿态模型建立”用代码实现，也比较简单
-

```
# writer: lihaonan_Bill
# date:  2022/3/14 19:03
"""
通过给定的数据利用几何关系计算出偏移角和偏移距离
               front car
node0  ------------m4------------node1
-                 m6                 -
-                 -                 -
-                 -                 -
-                 -                 -
-                 -                 -
-                 -                 -
-                 -                 -
node2  -------------m5-------------node3
              rear car
e.g. L01,L13分别表示node0到node1的距离，node1到node3的距离，其他的距离表示类似
角度表示形如Angle_204, Angle_205
计算姿态角和偏移距离和引导方向的详细算法见UWB编队解决方案
"""
import math


class Analyse:
    L01 = 95
    L23 = 95
    L46 = 95
```

```
    data = []

    @classmethod
    def cal(cls, L02, L12, L03, L13):
        """L13留作备用"""
        if L13:
            pass
        """Attitude"""
        Angle_201 = math.acos((L02 ** 2 + cls.L01 ** 2 - L12 ** 2) / (2 *
L02 * cls.L01))
        Angle_023 = math.acos((L02 ** 2 + cls.L23 ** 2 - L03 ** 2) / (2 *
L02 * cls.L23))
        Attitude = math.pi - (2 * math.pi - Angle_023 - Angle_201)
        """Distance"""
        L05 = (L02 ** 2 + (cls.L23 / 2) ** 2 - 2 * L02 * (cls.L23 / 2) *
math.cos(Angle_023)) ** 0.5
        Angle_205 = math.acos((L02 ** 2 + L05 ** 2 - (cls.L23 / 2) ** 2) /
(2 * L02 * L05))
        Angle_504 = Angle_201 - Angle_205
        L45 = (L05 ** 2 + (cls.L01 / 2) ** 2 - 2 * L05 * (cls.L01 / 2) *
math.cos(Angle_504)) ** 0.5
        """Orientation"""
        Angle_045 = math.acos(((0.5 * cls.L01) ** 2 + L45 ** 2 - L05 ** 2) /
(2 * (0.5 * cls.L01) * L45))

        if abs(Angle_045 - math.pi / 2) < 0.01:  # 在正后方时三角形456不存在
            L56 = L45 - cls.L46
            DeltaAngle = Attitude
        else:
            """ChasingDis"""
            Angle_546 = abs(math.pi / 2 - Angle_045)
            L56 = (L45 ** 2 + cls.L46 ** 2 - 2 * L45 * cls.L46 *
math.cos(Angle_546)) ** 0.5
            """ChasingAngle"""
            Angle_max = math.pi - math.acos((L56 ** 2 + cls.L46 ** 2 - L45
** 2) / (2 * L56 * cls.L46))
            DeltaAngle = (Attitude - (abs(math.pi / 2 - Angle_045) /
(math.pi / 2 - Angle_045)) * Angle_max)
        """把数据存入data"""
        cls.data.append(Attitude / math.pi * 180)  # 把角度从弧度制转为角度制
        cls.data.append(L45)
        cls.data.append(Angle_045 / math.pi * 180)  # 把角度从弧度制转为角度制
        cls.data.append(L56)
        cls.data.append(DeltaAngle / math.pi * 180)  # 把角度从弧度制转为角度制

    @classmethod
    def getData(cls):
        """
        拿出数据之后清除原来数据
        Attitude:[-180,180]
        当Attitude>0, L01和L23在左边相交
        当Attitude<0, L01和L23在右边相交
        Distance:[0,+Infinite]
        Orientation:[0,180]
        [0,90]:后车在前车左后方
        [90,180]:后车在前车右后方
        ChasingDis
        ChasingAngle:[-180,180]
```

```
        不需要担心原来不存在数据，因为执行完cal()之后一定有数据，也就是说不存在线程安全的问题
        """
        temp = cls.data[0:5]
        for i in range(len(temp)):
            cls.data.remove(cls.data[0])
        return temp


if __name__ == "__main__":
    Analyse.cal(167.2, 157.4, 230.9, 179.9)
    print(Analyse.getData())
```

控制算法--加入pid

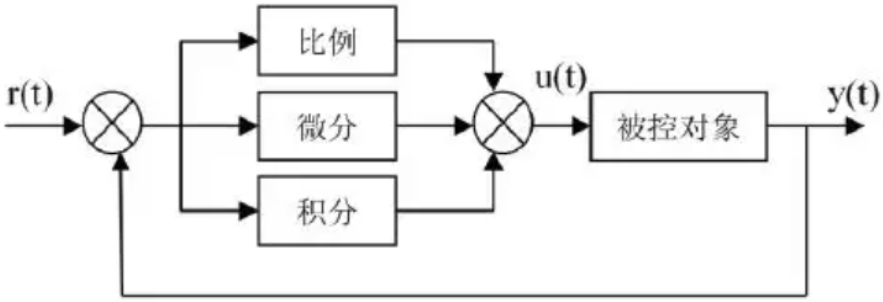
- pid的概念
  - 所谓pid其实就是在闭环控制的基础上增加了一些影响因子

$$U(t) = Kp(err(t) + \frac{1}{T_1})\int err(t)dt + \frac{T_D derr(t)}{dt}$$

而我们显然是需要进行离散状态下的转换，也就是下式

$$u(k) = K_p e(k) + \frac{K_p T_d}{Ti}\sum_{n=0}^{K} e(n) + \frac{K_p T_d}{T}(e(k) - e(k-1))$$

至于各项系数的确定，则则需要大量的实验和经验来判断才能做到丝般顺滑的效果




```
# writer: lihaonan_Bill
# date:  2022/3/14 18:53
# -----------包的导入-----------------
from smbus import SMBus
from PCA9685 import PWM
import time
from Analyse import Analyse


class CarControl:
    # ---------相关参数设定及初始化---------
```

```
    fPWM = 50
    i2c_address = 0x40
    channel = 1
    bus = SMBus(1)  # Raspberry Pi revision 2
    pwm = PWM(bus, i2c_address)  # 将频率转换为脉冲时间 20ms周期 = 频率为50Hz
    pwm.setFreq(fPWM)
    Divide = []  # 用于微分控制算法,第一项是历史值，第二项是实时值
    Integration = []  # 用于积分控制算法，下标越小历史越久远，也就是说最后一项是最新的
数据

    @classmethod
    def setTurn(cls, duty):
        """
        -------------------------------------
        函数名称：setTurn()
        函数功能：通过输入参数调用pca9685控制转向
        输入参数：duty，Initial_Value  输出参数:
        编制人：周峰立       编写日期：2021.02.04
        -------------------------------------
        """
        cls.pwm.setDuty(0, duty)

    @classmethod
    def setForward(cls, duty):
        """
        -------------------------------------
        函数名称：setForward()
        函数功能：通过输入参数调用pca9685控制前进
        输入参数：duty        输出参数:
        编制人：周峰立             改进日期：2021.02.04
        -------------------------------------
        """
        cls.pwm.setDuty(4, duty)  # 通过pwm脉冲来改变小车的状态

    @classmethod
    def AdaptiveShift(cls, Distance):  # 自适应变速(pid)
        """
        function:adjust the velocity according to the distance
        :param Distance:
        :return:
        """

        """判断微分和积分能不能用"""
        if len(cls.Divide) < 2 or len(cls.Divide) < 2:  # 必须有两个及以上的有
效数据
            return
        AmplificationFactor = 200
        DeltaPWM = 1
        PausePWM = 8
        Kp = 1 / AmplificationFactor
        Ki = 1 / 1000
        Kd = 1 / 15
        Pid = Kp * Distance + Ki * sum(cls.Integration) + Kd *
(cls.Divide[1] - cls.Divide[0])
        if Pid > 1:  # 距离太大，最大速度
            CarControl.setForward(PausePWM + DeltaPWM)
        elif Pid < 0:
            CarControl.setForward(PausePWM)
```

```
        else:  # 距离适宜，自适应变速
            CarControl.setForward(Pid * DeltaPWM + PausePWM)

    @classmethod
    def AdaptiveTurn(cls, Attitude, flag):  # 控制姿态，仅在后车中心处于前车中垂线
附近时使用
        """

        :param Attitude:
        :param flag: 三种模式
        1.转向角度与Attitude成正比
        2.大角度转弯，不具有自适应性
        3.转向角度与Orientation成正比

        :return:
        """
        if flag == 1:  # 自适应转弯
            if abs((-1) * (Attitude / 180)) * 4 < 2.5:
                CarControl.setTurn((-1) * (Attitude / 180) * 4 + 7.8)
            else:
                CarControl.setTurn((-1) * (Attitude / 180) * 2.5 + 7.8)
        elif flag == 2:  # 大角度转弯
            CarControl.setTurn((-1) * (Attitude / abs(Attitude)) * 2.5 +
7.8)
        else:
            if abs((-1) * (Attitude / 90)) * 8 < 2.5:
                CarControl.setTurn((-1) * (Attitude / 90) * 8 + 7.8)
            else:
                CarControl.setTurn((-1) * (Attitude / 90) * 2.5 + 7.8)

    @classmethod
    def carMove(cls, Attitude, Distance, Orientation, ChasingDis,
ChasingAngle):
        MinDis = Analyse.L46
        if Attitude:
            pass
        if Orientation:
            pass
        if Distance <= MinDis:  # 在这种情况下可以保证ChasingDis>
            cls.setForward(8)
            cls.Divide.clear()
            cls.Integration.clear()
        else:
            # 对历史数据的增和删
            if len(cls.Divide) > 2:
                cls.Divide.remove(cls.Divide[0])
            cls.Divide.append(ChasingDis)
            if len(cls.Integration) > 0:
                cls.Integration.append(ChasingDis)
            # 油门
            cls.AdaptiveShift(ChasingDis)
            # 方向
            cls.AdaptiveTurn(ChasingAngle, 1)


if __name__ == "__main__":
    while True:
        order = input("input,there must be a blank!!!")
```

```
        start = time.time()
        if order[0] == "0":
            CarControl.setTurn(int(order[2:]))
        elif order[0] == "4":
            CarControl.setForward(int(order[2:]))
        else:
            pass
        finish = time.time()
        print("time:", finish - start)
```

--李浩楠

--2022/4/16